COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/18/2026	**DATE SUBMITTED:**	09/18/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 9.1 [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Table 9.1

```cpp
#include <iostream>
#include "BST.h"

int main(){
    //root node:
    Tree<char> tree('A');

    //insert child nodes of A
    Tree<char>* B = tree.addChild('B');
    Tree<char>* C = tree.addChild('C');
    Tree<char>* D = tree.addChild('D');
    Tree<char>* E = tree.addChild('E');
    Tree<char>* F = tree.addChild('F');
    Tree<char>* G = tree.addChild('G');

    //child of D
    Tree<char>* H = D->addChild('H');

    //children of E
    Tree<char>* I = E->addChild('I');
    Tree<char>* J = E->addChild('J');

    //children of F
    Tree<char>* K = F->addChild('K');
    Tree<char>* L = F->addChild('L');
    Tree<char>* M = F->addChild('M');

    //child of G
    Tree<char>* N = G->addChild('N');

    //children of J
    Tree<char>* P = J->addChild('P');
    Tree<char>* Q = J->addChild('Q');

    //traversing
    std::cout<< "preorder: ";
    tree.preOrder();
    std::cout<<std::endl;

    std::cout<< "inorder: ";
    tree.inOrder();
    std::cout<<std::endl;

    std::cout<< "postorder: ";
    tree.postOrder();
    std::cout<<std::endl;
```

HEADER FILE

```cpp
#ifndef TREE_H
#define TREE_H

#include <iostream>

template <typename T>
class Tree
{
private:
    // create a key
    T key;
    Tree<T>* firstChild;
    Tree<T>* nextChild;

public:
    // constructor
    Tree(T value);

    // add child
    Tree<T>* addChild(T value);

    // traversing
    void preOrder();
    void postOrder();
    void inOrder();

    // searching
    void findData(int CHOICE, T KEY);
};

// Constructor
template <typename T>
Tree<T>::Tree(T value)
{
    key = value;
    firstChild = nullptr;
    nextChild = nullptr;
}
// Add child
template <typename T>
Tree<T>* Tree<T>::addChild(T value)
{
    Tree<T>* newNode = new Tree<T>(value);
```

```
    if (firstChild == nullptr){
        firstChild = newNode;
    } else {
        Tree<T>* temp = firstChild;
        while (temp->nextChild != nullptr){
            temp = temp->nextChild;
        }
        temp->nextChild = newNode;
    }

    return newNode;
}


// Preorder
template <typename T>
void Tree<T>::preOrder(){
    // current node
    std::cout << key << " ";
    // first child
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    if(nextChild != nullptr){
        nextChild->preOrder();
    }
}


// Inorder
template <typename T>
void Tree<T>::inOrder(){
    // first child
    if(firstChild != nullptr){
        firstChild->inOrder();
    }
    // current node
    std::cout << key << " ";
    if(nextChild != nullptr){
        nextChild->inOrder();
    }
}


// Postorder
```

```cpp
// Postorder
template <typename T>
void Tree<T>::postOrder(){
    Tree<T>* child = firstChild;

    while(child != nullptr){
        child->postOrder();
        child = child->nextChild;
    }
    // current node
    std::cout << key << " ";
}
```

Table 9.2

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2

M	0	2
N	0	2
P	0	3
Q	0	3

Table 9.3

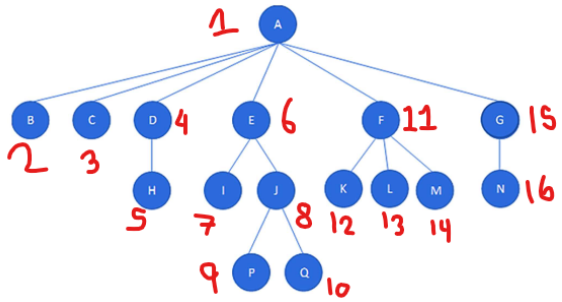
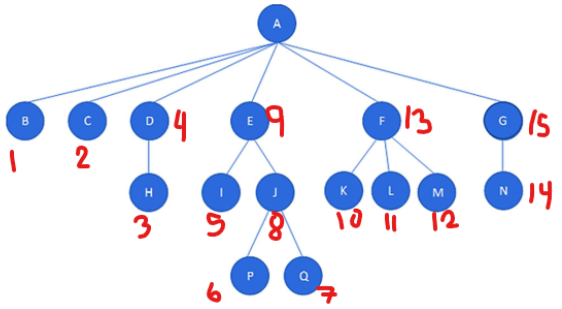
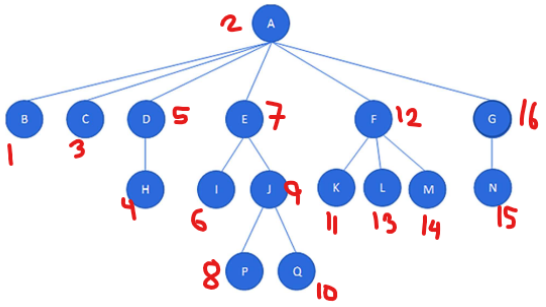
Pre-Order	
Post-Order	
In-Order	

Table 9.4

Pre-Order

```
// Preorder
template <typename T>
void Tree<T>::preOrder(){
    // current node
    std::cout << key << " ";
    // first child
    if(firstChild != nullptr){
        firstChild->preOrder();
    }
    if(nextChild != nullptr){
        nextChild->preOrder();
    }
}
```

```
preorder: A B C D H E I J P Q F K L M G N
```

The resulting output of a pre-order binary tree traversal containing letters A to Q, given that the print command is executed prior to both the null-pointer check and the child node pointers.

Post-Order

```
// Postorder
template <typename T>
void Tree<T>::postOrder(){
    Tree<T>* child = firstChild;

    while(child != nullptr){
        child->postOrder();
        child = child->nextChild;
    }
    // current node
    std::cout << key << " ";
}
```

```
postorder: B C H D I P Q J E K L M F N G A
```

The resulting output of a post-order binary tree traversal for letters A to Q, noting that the print operation is properly executed after the conditional check and child pointers.

In-Order

```
// Inorder
template <typename T>
void Tree<T>::inOrder(){
    // first child
    if(firstChild != nullptr){
        firstChild->inOrder();
    }
    // current node
    std::cout << key << " ";
    if(nextChild != nullptr){
        nextChild->inOrder();
    }
}
```

```
inorder: B C H D I P Q J E K L M F N G A
```

The resulting output of a binary tree pre-order traversal for letters A to Q, given that the print statement is positioned in between the two conditional checks and the child pointers during the traversal process.

Table 9.5

```
preorder: A B C D H E I J P Q F K L M G N
inorder: B C H D I P Q J E K L M F N G A
postorder: B C H D I P Q J E K L M F N G A

Find H using preorder: H was found!
Find I using inorder: I was found!
Find Q using postorder: Q was found!
Find Z using preorder:
```

The resulting output of the newly introduced findData function, which accepts Choice and key parameters to specify the traversal method and the target letter being searched for within the tree.

Table 9.6

```
Find O using preorder: O was found!
Find O using inorder: O was found!
Find O using postorder: O was found!
```

The resulting output of adding a new leaf node with the data value "O" as a child of node G, followed by locating it using the findData function. This demonstrates that while different traversal methods navigate the tree differently, they all successfully reach the node and return the identical value of "O."

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Step 1

```cpp
#include <iostream>
#include "supplementary.h"

int main() {

    // Step 1: Implement a binary search tree with the given values
    // Values: 2, 3, 9, 18, 0, 1, 4, 5
    // First value (2) becomes the root; the rest are inserted after.
    BinarySearchTree<int>* root = new BinarySearchTree<int>(2);

    root->insert(3);
    root->insert(9);
    root->insert(18);
    root->insert(0);
    root->insert(1);
    root->insert(4);
    root->insert(5);

    // Step 3: Compare the different traversal methods

    std::cout << "In-order Traversal: ";
    root->inOrder();
    std::cout << std::endl;

    std::cout << "\nPre-order Traversal: ";
    root->preOrder();
    std::cout << std::endl;

    std::cout << "\nPost-order Traversal: ";
    root->postOrder();
    std::cout << std::endl;

    return 0;
}
```

Header File

```cpp
#ifndef SUPPLEMENTARY_H
#define SUPPLEMENTARY_H

#include <iostream>

template <typename T>
class BinarySearchTree{
    private:
    //create a Key
    T key;
    //pointers for left and right
    BinarySearchTree<T>* left;
    BinarySearchTree<T>* right;

    public:
    //constructor
    BinarySearchTree(T value);

    // insert left and insert right
    void insert(T value);
    //traversing
    void preOrder();
    void inOrder();
    void postOrder();
    //searching



};
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key=value;
    left = nullptr;
    right = nullptr;
}
template <typename T>
void BinarySearchTree<T>::insert(T value){
```

```
    //left or right

    //go left if less than key insert to left

    //go right if greater than key insert to right
    if (value < key){
        if (left == nullptr){
        left = new BinarySearchTree<T>(value);

        }else{
            left -> insert(value);
        }
    }else if(value > key){
        if (right == nullptr){
        right = new BinarySearchTree<T>(value);

        }else{
            right -> insert(value);
            }
        }
}


template <typename T>
void BinarySearchTree<T>::preOrder(){

    std::cout<<key<<" ";
    if(left != nullptr){
        left->preOrder();
    }
    if(right !=nullptr){
        right->preOrder();
    }
}
```

```cpp
template <typename T>
void BinarySearchTree<T>::inOrder(){

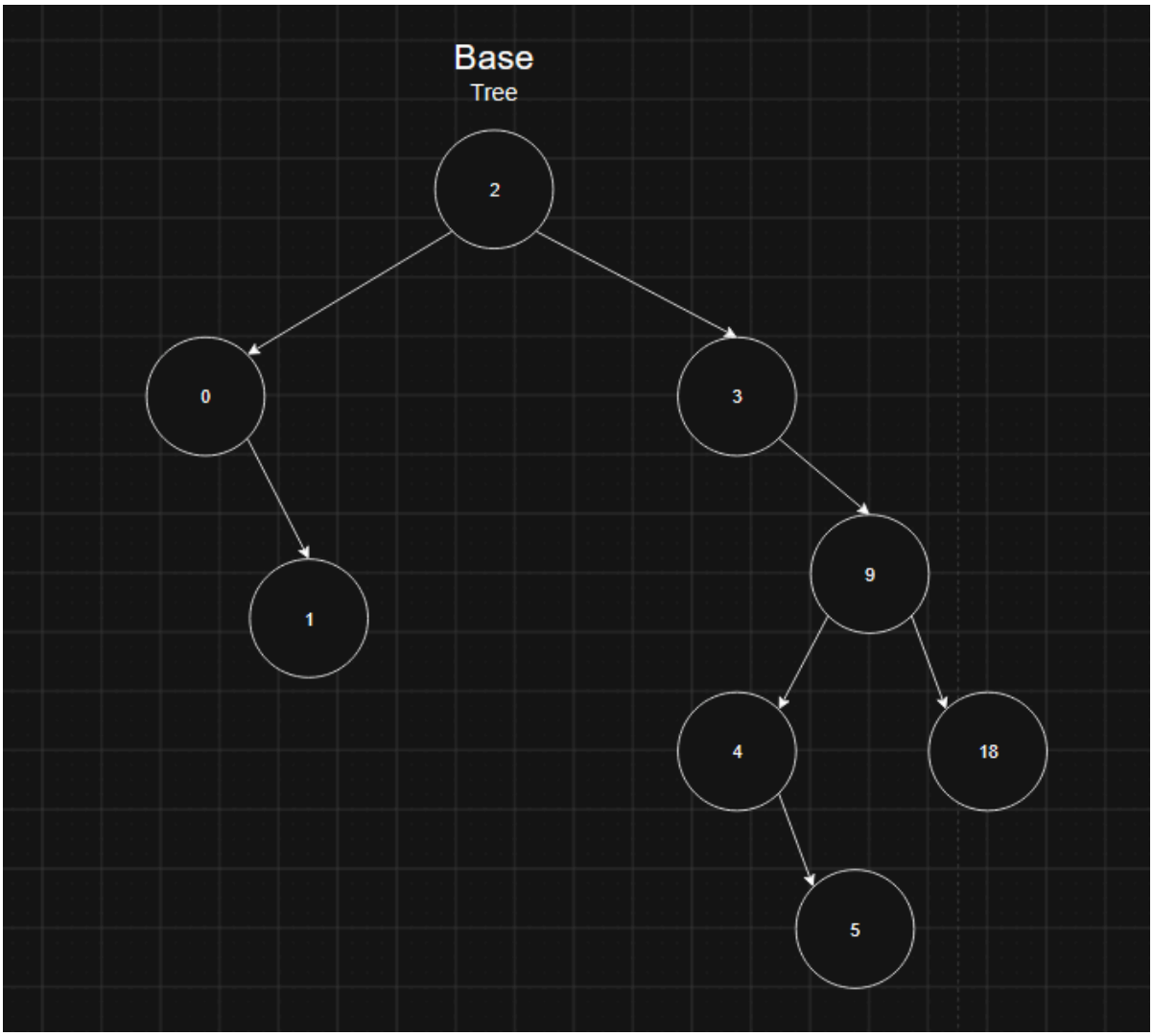
    if(left != nullptr){
        left->inOrder();
    }
    std::cout<<key<<" ";
    if(right !=nullptr){
        right->inOrder();
    }
}

template <typename T>
void BinarySearchTree<T>::postOrder(){
    if(left != nullptr){
        left->postOrder();
    }

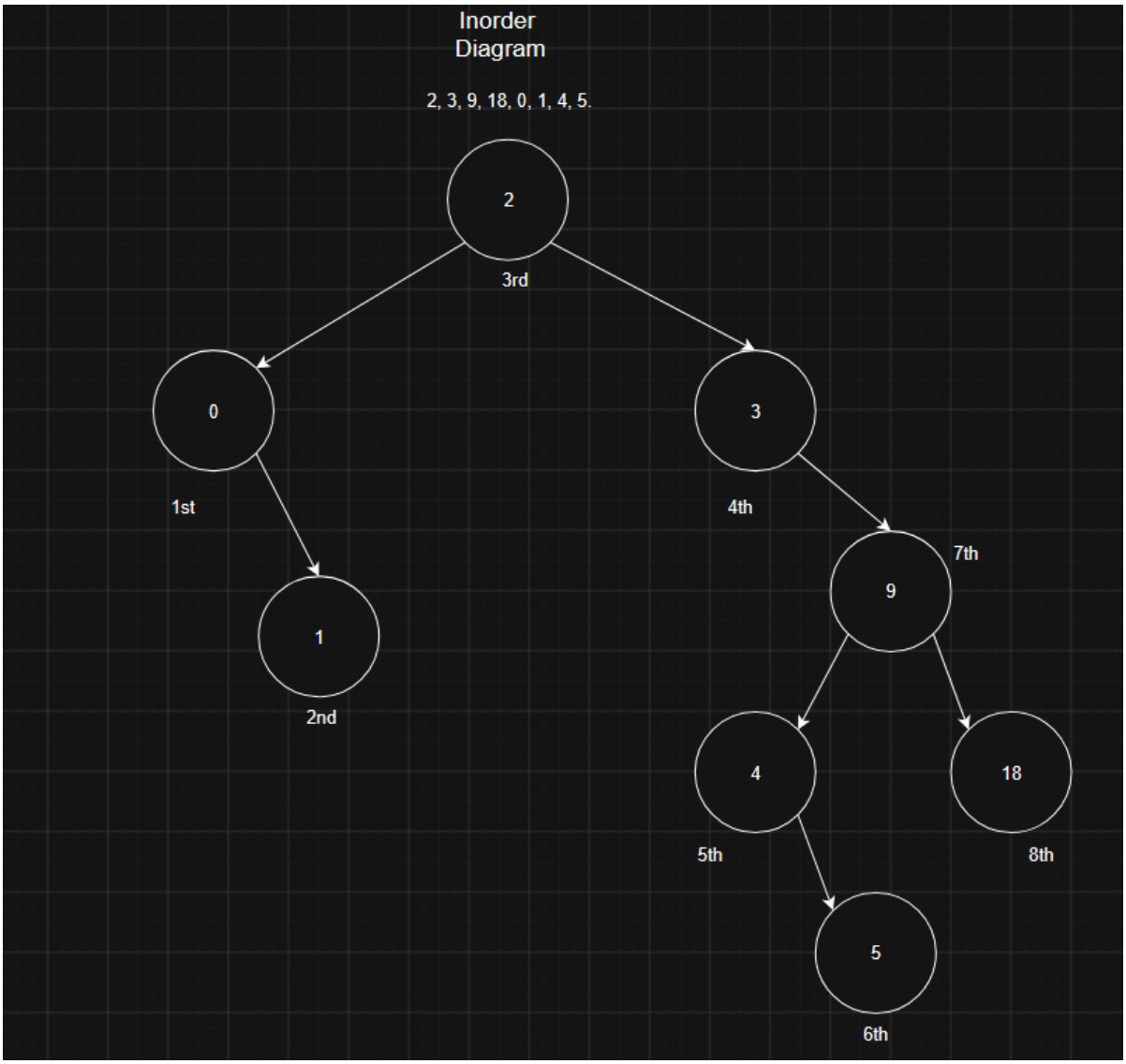
    if(right !=nullptr){
        right->postOrder();
    }
    std::cout<<key<<" ";
}
#endif
```

Step 2

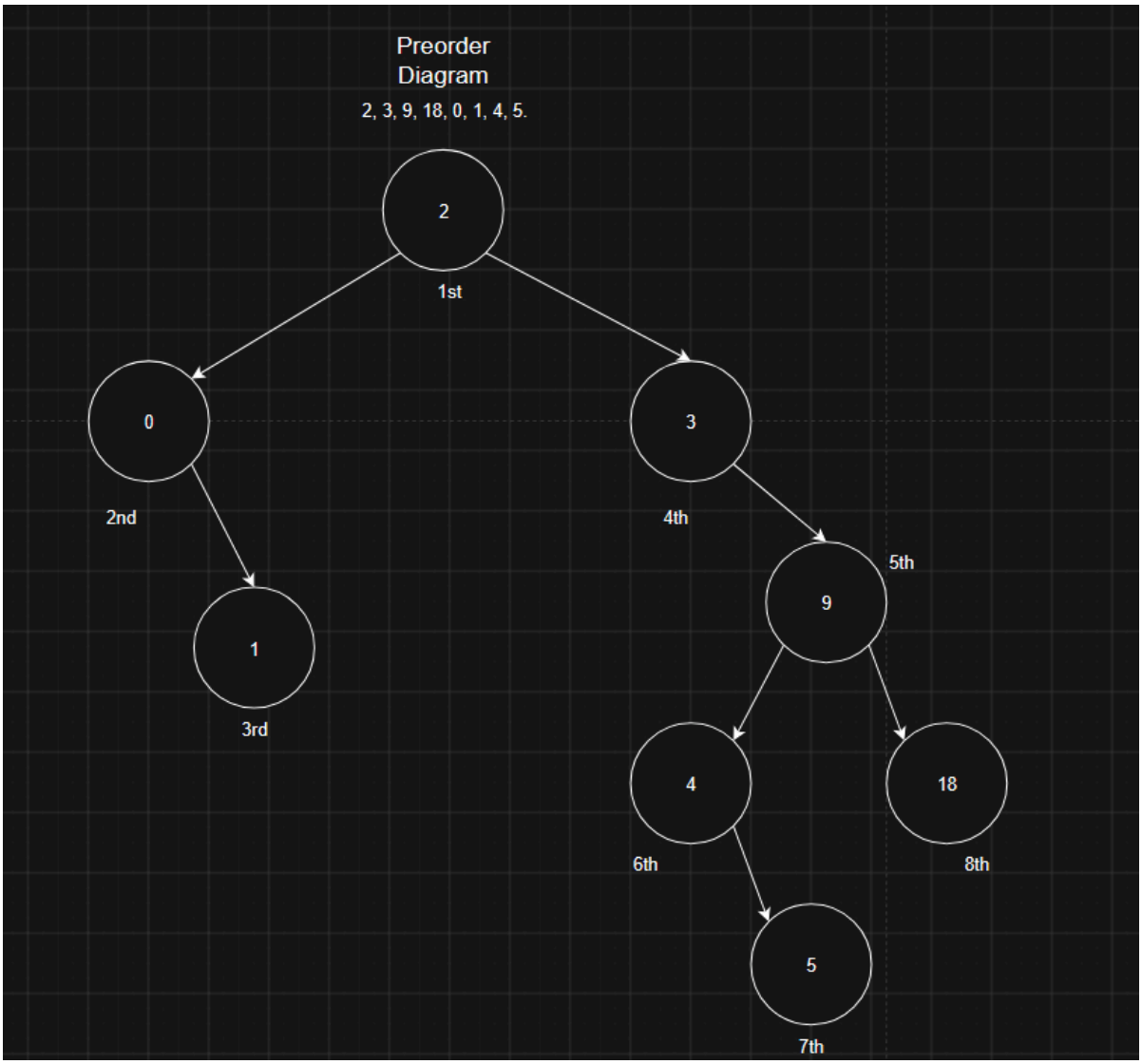
Base Tree



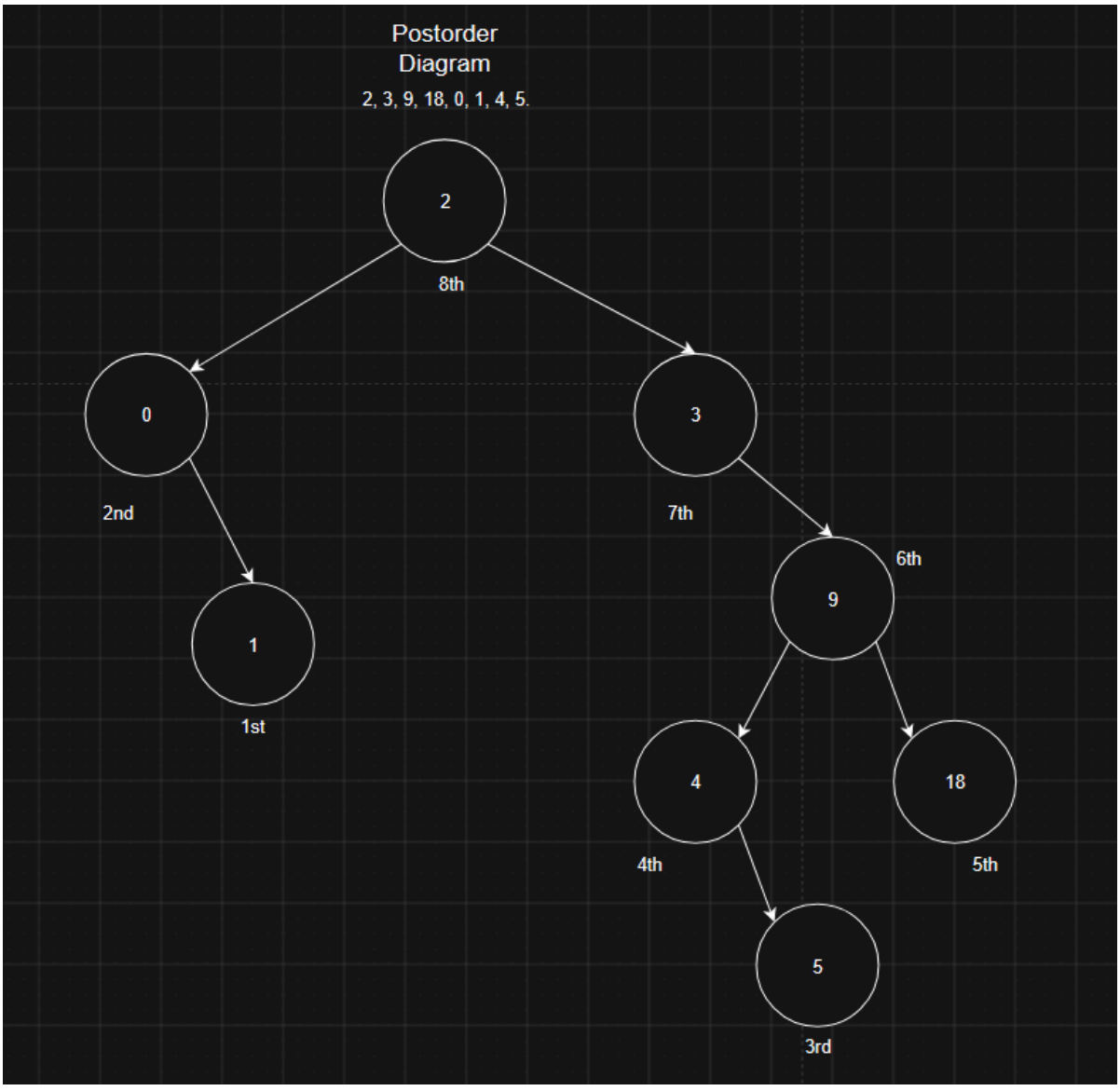
Inorder Diagram



Preorder Diagram



Postorder Diagram



Step 3

```
In-order Traversal: 0 1 2 3 4 5 9 18

Pre-order Traversal: 2 0 1 3 9 4 5 18

Post-order Traversal: 1 0 5 4 18 9 3 2
```

In-order gives the sorted sequence, while pre-order and post-order are both different from each other and from in-order, pre-order visits the root first (2 leads), and post-order visits it last.

Conclusion

The activity focuses on understanding the fundamentals and implementation of binary trees using different scenarios and sets of values. By learning how binary trees work, we can create and organize trees with different root values while also applying various traversal methods to access child nodes in different orders. Furthermore, understanding the basics of binary trees helps us implement searching techniques to locate specific values within the tree.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.